

BOOST: Concurrent Access to Host Memory and HBM to Accelerate LLM Inference

Anish Saxena[†]
Georgia Tech
Atlanta, USA

Jae Hyung Ju[†]
Georgia Tech
Atlanta, USA

Hritvik Taneja
Georgia Tech
Atlanta, USA

Po-An Tsai
Nvidia Research
Westford, USA

Aamer Jaleel
Nvidia Research
Westford, USA

Christos Kozyrakis
Nvidia Research & Stanford
Santa Clara, USA

Moinuddin Qureshi
Georgia Tech
Atlanta, USA

Abstract

GPU memory bandwidth and capacity limit throughput in large language model (LLM) inference. The GPU memory system consists of a primary tier of high-bandwidth memory (HBM) and a secondary tier of host memory connected via CPU-to-GPU interconnect. Current serving systems treat the tiers hierarchically: they serve exclusively from HBM when data fits, and otherwise prefetch data from host memory to HBM before use. In both cases, the host memory bandwidth is never well utilized. Prefetching expands capacity by utilizing host memory, but consumes HBM bandwidth for writes, reducing the bandwidth available for demand loads. We observe that fully utilizing both host and HBM bandwidth requires each wave of GPU threadblocks to access both tiers *concurrently* and in *proportion* to their bandwidth ratio. Existing bandwidth-proportional placement strategies fail to provide concurrency because they are not aware of GPU waves, and the large 2MB GPU page size.

This paper presents *BOOST*, the first runtime system that provides concurrent and proportional access to both GPU memory tiers, extracting the combined bandwidth of host memory and HBM for LLM inference *without kernel changes*. The key insight in *BOOST* is to use kernel access patterns to make page allocation and runtime data management *wave-aware*. For static model weights, *BOOST* applies modulo-based page placement that eliminates access-ratio variance; for dynamically provisioned attention key-value (KV) pairs, it makes the free KV page pool wave-aware. We integrate *BOOST* into vLLM and evaluate on a Grace Hopper system. At iso-batch size, *BOOST* improves Time-per-Output-Token (TPOT) by 4.3% over HBM-only serving, whereas prefetching degrades TPOT by 6%. In high-throughput serving, *BOOST* improves throughput by 31% on average, outperforming prefetching by 15%.

1 Introduction

The memory system of the GPU determines the processing speed of modern Large Language Models (LLMs) [2, 7, 33, 69]. The decode phase of LLM inference is autoregressive, where hundreds of gigabytes of model weights and attention key-value (KV) pairs are fetched from GPU memory at every iteration, making LLM decoding limited by memory bandwidth, and often capacity.

The GPU memory system consists of two tiers. The first tier is *High-Bandwidth Memory (HBM)*, the primary tier for serving memory requests. The second tier is *host* memory located at the CPU and connected to the GPU via a coherent *CPU-to-GPU (C2G)*

interconnect. Host memory bandwidth is lower than HBM bandwidth, but is often a significant fraction of it. For example, Nvidia’s Grace Hopper [16, 39] has host memory with approximately 9% to 11% of the HBM bandwidth depending on SKU. Therefore, the overall memory bandwidth of a system that utilizes both HBM and host memory is approximately 1.1x that of an HBM-only system.

Ideally, the system should exploit the combined bandwidth of both tiers, improving the *Time-Per-Output-Token (TPOT)* by up to 1.1x. The host tier also adds capacity to batch more requests and raise system throughput. For instance, if a system uses 20% of the HBM capacity to store KV cache, then 10% additional capacity from the host tier increases the KV space by 50%, allowing up to 1.5x more concurrent requests and up to 1.5x higher throughput.

Existing systems do not fully utilize the available memory bandwidth and capacity. Fig. 1-(a) shows that when HBM capacity is sufficient to hold the model weights and KV cache, data is served exclusively from HBM, leaving host memory idle. When the working set exceeds HBM capacity, as Fig. 1-(b) shows, current systems [9, 65] place a portion of the data in the host memory and migrate (prefetch) the data from host memory to HBM before use. However, migrating data from the host to HBM consumes HBM write bandwidth, reducing the available HBM read bandwidth that serve GPU demand loads. Even an ideal prefetcher that fully overlaps migration delivers less read bandwidth than HBM alone, because every prefetching incurs HBM writes. The fundamental problem is that current systems treat the two tiers as a hierarchy rather than peers, and data is directly served only from HBM.

To fully extract system-wide memory bandwidth, data accesses must satisfy two properties: *concurrency* and *proportionality*. The first property dictates that accesses must concurrently go to both tiers, even over short timescales. The second property dictates that accesses be split proportionally to the bandwidth ratio of each tier. For example, if host memory has 10% of HBM bandwidth, it should receive 10x fewer accesses than HBM. We note that violating the second property can cause slowdown compared to HBM-only serving. For example, if more than a proportional share of accesses go to host memory, the system would become bottlenecked by host bandwidth while HBM bandwidth remains underutilized.

Prior works on heterogeneous GPU memory [3, 4, 11] achieve proportionality by randomly assigning pages across tiers in proportion to their bandwidth ratio. Developed a decade ago for 4KB pages, these schemes obtained concurrency implicitly: a wave touched thousands of pages, so a proportional share reached the host tier within every wave. Modern GPUs use 2MB pages to limit TLB

[†] Equal contribution. Anish and Jae can be reached at {asaxena317,jhju}@gatech.edu

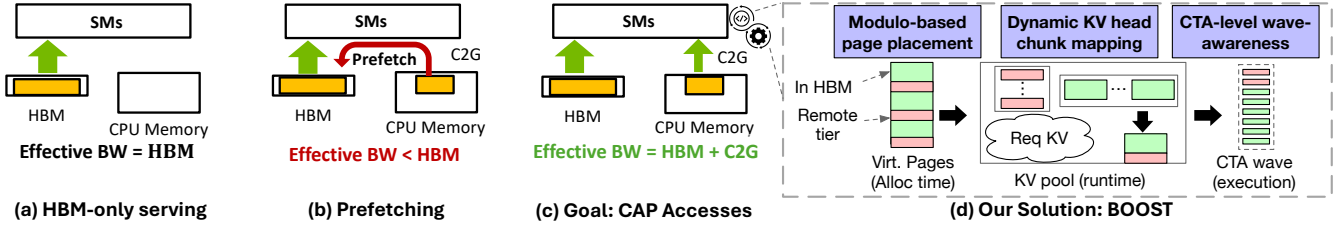


Figure 1: (a) LLM serving systems use HBM exclusively, leaving host CPU memory idle. (b) To improve serving capacity, current systems migrate (prefetch) data, degrading HBM demand bandwidth. (c) Our goal is to perform concurrent and proportional (CAP) accesses at each wave to fully utilize system bandwidth. (d) BOOST facilitates CAP accesses by making page placement and runtime data management aware of CTA-level kernel data access patterns.

pressure, so a wave touches only tens of pages, and random placement that is proportional on average deviates widely within a wave, which underutilizes the host tier or, worse, stalls on it and idles the HBM. Our evaluations show that such a random page allocation policy does not benefit LLMs. Deployed serving systems, meanwhile, use the host tier only through prefetching; serving inference directly from data placed across both tiers remains unexplored.

The goal of our work is to enable *Concurrent and Proportional* (CAP) memory accesses to both tiers of the GPU memory system, as shown in Fig. 1-(c). However, LLM serving systems are complex, with multiple abstraction layers between physical memory and the runtime data accessed by kernels, which render page-to-data mapping opaque to the kernel and complicate CAP accesses. Moreover, making low-level modifications to highly optimized kernels is both intrusive and impractical, as some kernels are closed source [12]. Ideally, we want to enable CAP without requiring kernel changes.

In this paper, we present *BOOST*, the first runtime system that enables CAP memory access, extracting the additive bandwidth of both host and HBM tier for LLM inference, *without* any kernel modifications. BOOST distributes data to both memory tiers, thus also increasing the usable memory capacity to maximize system throughput. The core idea behind BOOST is to leverage access patterns in key LLM kernels to guide page placement and runtime data management, facilitating CAP transparently to the kernel (Fig. 1-(d)). BOOST overcomes three key challenges.

First, page placement. Each wave (set of concurrently executing threadblocks) accesses tens of megabytes of data, spanning only tens of 2MB pages, so random allocation varies the access ratio across waves. We want *deterministic* access ratio to the two tiers even with a few pages. We find that kernels that use static data, such as model weights, access virtually contiguous memory during its waves. Therefore, we propose *Modulo-Based Page Placement (MPP)*: for an HBM-to-host bandwidth ratio of K , MPP allocates the first K pages to HBM and the $(K + 1)^{th}$ page to host memory. MPP enables CAP for static data even with a few 2MB pages.

Second, granularity. Kernel launch parameters (warps per threadblock and total threadblocks) vary significantly. We could guide MPP using data access patterns at the thread, warp, or threadblock level (also known as cooperative thread arrays, or CTAs). Our characterization on GH200 GPU reveals that CTA size shows little variance (4 or 8 warps) but warp specialization (different warps in a CTA access different tiers) is too coarse: 4-warp CTAs only support

bandwidth ratios in multiples of 0.25. Thread specialization is ineffective because memory divergence prevents request coalescing. Our second insight is to use *CTA-level wave-awareness* to guide data management, enabling CAP for typical LLM kernels.

Third, handling dynamic data. CTAs in attention kernels access dynamically allocated KV cache. By default, the serving system adds and removes KV pages [30] from the pool of free KV pages at runtime without being wave-aware. Over time, the dynamic allocation and freeing of KV pages make the mapping from virtual address to CTAs near-random, which no static placement can correct. Moreover, recent attention kernels [49, 61] dynamically partition the KV cache of each request into chunks when mapping KV to CTAs. Because the chunk boundaries change over time, it is challenging to find a static data placement across tiers that achieve wave-awareness at CTA-level. Our third insight is that, despite the dynamism, chunks often align with KV head and request boundaries due to the inherent parallelism. By mapping certain KV head of selected requests to the host tier, we can approximately maintain the desired memory ratio for attention kernels.

We integrate BOOST into vLLM and evaluate three LLMs on a Grace Hopper system with a host to HBM bandwidth ratio of 10%. In iso-batch low-latency serving, BOOST improves TPOT by 4.3% over HBM-only serving, while prefetching degrades TPOT by 6%. In high-throughput serving, BOOST improves throughput by 31% on average: the added host capacity provides most of this gain, and concurrent access contributes a further 4% that no capacity-only mechanism provides. BOOST outperforms prefetching by 15% owing to its higher demand load bandwidth.

Overall, our paper makes the following contributions:

- We make the key observation that performance of LLM serving can be improved by exploiting the bandwidth of both HBM and host memory. To do so, we need *Concurrent and Proportional* (CAP) accesses to both tiers. We show that prior placement schemes provide proportionality but lose concurrency when GPUs moved from 4KB to 2MB pages.
- We present BOOST, the first solution that enables CAP accesses for LLM serving through wave-aware modulo page placement for static model weights and wave-aware runtime management for dynamic data, *without kernel changes*.
- We show that BOOST provides a 4.3% TPOT improvement at iso-batch and a 31% average throughput speedup over HBM-only serving (with 4% owing to concurrent accesses to both tiers), outperforming prefetching by 15%.

2 Background and Motivation

2.1 Large Language Model Inference

Large language models (LLMs) dominate modern GPU workloads. The decode phase generates tokens autoregressively and reloads the model weights every iteration, so it dominates end-to-end latency, increasingly so for reasoning models [36]. Decoding is memory bandwidth-bound: attention and MLP kernels for dense LLMs, and expert kernels for Mixture-of-Experts (MoE) models, repeatedly fetch the KV cache and model weights. DeepSeek R1 [19], for instance, holds 670GB of weights, and its KV cache adds 1GB per user at 32K context. Serving systems [15, 17, 31, 41, 68] layer kernel and scheduling optimizations [5, 30, 49, 61] between a kernel and its demand loads, so the memory system observes complex access patterns. We target the bandwidth-bound decode phase under prefill-decode (PD) disaggregation [69], in both low-latency (minimizing TPOT) and high-throughput (maximizing TPS) serving; the latter is also capacity-bound.

2.2 Kernel Execution on GPUs

GPUs execute kernels using a hierarchical scheduling model that maps computation to the streaming multiprocessors (SMs). The kernel is divided into many threadblocks (also known as cooperative thread arrays, or CTAs), depending on problem size. Each CTA comprises several warps of 32 threads. The GPU scheduler assigns multiple CTAs to SMs depending on available resources (register files, shared memory, and warps), and once assigned, a CTA occupies SM resources until it completes execution. Within each active threadblock, the SM can dynamically schedule warps in and out of execution based on their readiness. LLM kernels can launch thousands of threadblocks, and typically, hundreds of threadblocks are active at each cycle. We refer to the set of active threadblocks across all SMs in the GPU as a *wave*.

2.3 GPUs with Heterogeneous Memory Systems

GPU memory systems are heterogeneous, comprising two memory tiers: the primary local memory (typically HBM) and a secondary host memory tier accessible through CPU-to-GPU (C2G) interconnect [21]. As shown in Fig. 2, GPUs have access to CPU memory through cache-coherent C2G links with load-store semantics. However, the secondary tier provides only a fraction of HBM’s bandwidth. We define α as the ratio of C2G read bandwidth to HBM bandwidth. In Nvidia’s Grace Hopper (GH200) system [16, 39], α ranges from 9% to 11% as C2G read bandwidth is 450GB/s and HBM bandwidth ranges between 4TB/s to 4.9TB/s depending on the SKU. Unlike traditional PCIe-connected systems where one CPU drives multiple GPUs, we focus on tightly-coupled CPU-GPU systems like GH200 with a measured bandwidth ratio of 10% (§3 to §5). We further analyze future systems with different α (§5.12 to §6.1).

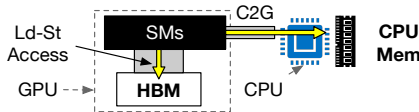


Figure 2: The GPU memory system connects HBM and host CPU memory (attached via C2G links) to the GPU in parallel.

2.4 Requirements for Additive Bandwidth

To fully utilize the HBM+C2G bandwidth $(1 + \alpha)$, accesses from SMs must satisfy two properties. *Proportionality* requires that α ratio of accesses reach host relative to HBM [3, 11]. *Concurrency*, a property we identify in this work, requires that the accesses within *every* wave reach both tiers in proportion. We show that violating either property impedes performance. Existing schemes which use host tier are either *migration-based* that move data to HBM before use, or *placement-based* that serve data from the tier where it resides.

2.5 Page Migration Schemes

Migration-based schemes stage data from the host tier in HBM before the SMs consume it, in two forms: prefetching in serving systems, and fault-driven migration in CUDA unified memory.

Prefetching. When the model and KV cache exceed HBM capacity, serving systems offload data to host memory and prefetch it to HBM before use [9, 24, 43, 55, 65]. Prefetching enables larger workloads to fit in the system but degrades demand bandwidth. Each byte read from host memory incurs a write to HBM, consuming bandwidth on the HBM interface. The writes to HBM are recurrent, as the offloaded data exceeds the HBM staging buffer capacity, so staged bytes are evicted after use and fetched again at the next decode iteration. The writes reduce effective demand bandwidth from the ideal $(1 + \alpha)$ to $(1 - \alpha)$; even an ideal prefetcher that fully overlaps migration with computation cannot exceed this bound. In contrast, concurrently loading from both tiers attains $(1 + \alpha)$ demand bandwidth, a $(1 + \alpha)/(1 - \alpha)$ advantage over any prefetcher. At $\alpha = 10\%$, this benefit is 22%, and grows with increasing α (§6.1).

Impact of Prefetching on Demand Bandwidth. Fig. 3 shows the system bandwidth utilization for Llama-3.3 70B running on a Grace Hopper system using vLLM [55] (evaluation details in §5). The serving system offloads one out of every K layers and prefetches it asynchronously before use, where $1/K$ approximates the bandwidth ratio required to hide the migration latency from the host tier to HBM. Prefetching to HBM leads to two problems: (i) the migration degrades the HBM read bandwidth available for demand accesses (by 211GB/s in Fig. 3-left), and (ii) staging requires a buffer in HBM, marginally reducing usable capacity. To maximize performance, a system must launch demand accesses (not prefetches) to both memory tiers, rendering data migrations unnecessary.

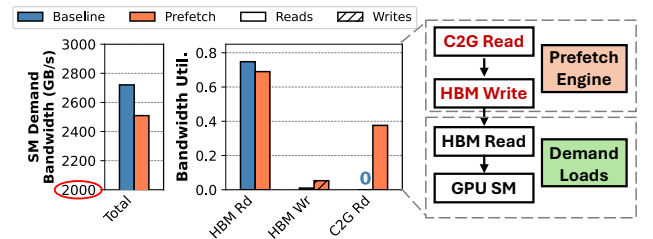


Figure 3: Left: Demand bandwidth to the SM in HBM-only baseline and prefetching for Llama-3.3 70B on GH200, both at batch size of 10 (details in §5). Middle: Breakdown of system bandwidth utilization. Right: Lifecycle of offloaded data. The prefetch engine migrates data from the C2G tier to HBM before use, reducing demand bandwidth available to the SMs.

CUDA Unified Memory. CUDA performs automatic migration via `cudaMallocManaged` that moves pages on faults toward the accessing processor to optimize locality. Unfortunately, once the working set (model+KV state) exceeds HBM, such a demand-based migration demotes the HBM into a page cache which gets thrashed on every iteration, because each page access brings in the faulted page and evicts another page. This makes decoding effectively a sequential on-demand fetch of non-HBM-resident pages, degrading performance significantly—we measure a $5\times$ throughput degradation (evaluations in §5.7). The migrations also consume HBM write bandwidth similar to prefetching. Both migration schemes serve every byte from HBM and do not issue demand loads to the host tier, so they provide neither proportionality nor concurrency.

2.6 Page Placement Schemes

Placement schemes instead treat the two tiers as peers, and data resides where it is placed while the SMs issue demand loads to both tiers, so no migration occurs. Note that serving systems use the host tier only through prefetching (Fig. 3). No prior work applies placement-based serving to LLM inference. We implement both schemes of this family and present its first characterization.

Bandwidth-Aware Page Placement. Prior works on data management for heterogeneous memory [3, 4, 11] aim to achieve proportionality through driver-level bandwidth-aware page placement, which distributes pages so a randomly selected α fraction resides in the host tier (BAPP-R). BAPP-R was developed a decade ago for GPUs that used small pages (e.g., 4KB) and obtained concurrency implicitly, because a wave typically accesses tens of megabytes and therefore touches thousands of 4KB pages. However, modern GPUs use 2MB pages to limit TLB pressure, and BAPP-R fails to achieve concurrency even with bandwidth-proportional placement. No implementation of BAPP-R exists for modern GPUs or systems, so we implement it ourselves. To characterize placement, we use a controlled microbenchmark: an add kernel that streams 1GB through thousands of CTAs across several waves, emulating the access pattern of LLM attention, expert, and MLP kernels (§3.2) with trivial compute, so any bandwidth loss is attributable to placement alone.

Impact of BAPP-R on Demand Bandwidth. Fig. 4 shows that BAPP-R degrades demand bandwidth to SMs by 115GB/s (3.5%) below the HBM-only baseline, against an ideal improvement of 10.4%. We find that BAPP-R guarantees proportional access only on average over the full kernel, not within each wave, which creates variable bandwidth ratios *across waves*. To illustrate, consider a biased coin with $P(\text{heads}) = 10\%$: across hundreds of flips roughly 10% are heads, but windows of few tens of flips deviate significantly. BAPP-R behaves the same. Across the full kernel (hundreds of 2MB pages) the placement ratio is $10\% \pm 1.5\%$ over 100 kernel instances; within a wave it varies far more, $10\% \pm 3.6\%$ (over 2,000 waves). The variance is asymmetrically harmful. Waves that over-access HBM underutilize the C2G link, and, more critically, waves that over-access the host tier stall on its lower bandwidth and underutilize HBM, degrading overall system bandwidth utilization. Reverting to 64KB pages does not work because the resulting TLB misses degrade HBM bandwidth below 1000GB/s, less than one-third of its maximum bandwidth (not shown in figure). Deterministic page interleaving could fix this randomness-induced variance.

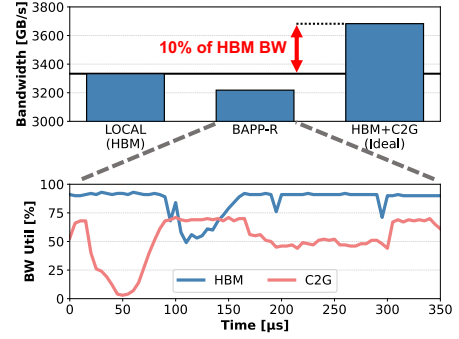


Figure 4: Top: Bandwidth usage of random page placement (BAPP-R) over HBM-only accesses. BAPP-R incurs a 3.5% lower usage while the ideal bandwidth improvement is 10.4%. Bottom: Bandwidth utilization of HBM and C2G links over time for BAPP-R. The temporal variance in bandwidth utilization degrades system bandwidth utilization.

CUDA does not expose bandwidth-aware allocation: `cudaMalloc` allocates only on HBM, the Virtual Memory Management (VMM) API provides full manual control but no automatic bandwidth-aware algorithm, and NVIDIA recommends manually partitioning data using `cudaMemSet` APIs. We implement bandwidth-aware page placement with the VMM API, but find that it degrades performance compared to our Linux-API-based design (details in §4.4).

Linux Weighted Interleaving. The Linux kernel supports deterministic weighted interleaving of pages across memory tiers via the `set_mempolicy()` interface [28], which places pages across NUMA nodes proportionally to relative tier bandwidth, typically for CPU workloads. The interleaving is agnostic to the consumer: pages stripe across the address range with no knowledge of which thread—or, on a GPU, which kernel or CTA—consumes which page. The striping can cross kernel boundaries, so an individual kernel may receive a host share above α , underutilizing HBM and incurring slowdown. Moreover, data a CTA reads over its lifetime need not align to 2MB pages—on the Llama-3.3 down-projection, one page holds 73 of the 128 columns a CTA computes—so the footprint of a CTA spans 1.75 pages, and when one page is host-resident, the host loads stall the SM and delay its co-issued HBM loads. Finally, striping is fixed at allocation and cannot maintain the α ratio for the KV pages that the serving system provisions and frees at runtime. As a result, weighted interleaving fails to improve system bandwidth utilization. Because no implementation exists for GPU memory, we implement it ourselves and evaluate it in §5.7.

2.7 Goal of Our Work

Table 1 compares existing schemes and outlines their key limitations. No existing scheme provides per-wave concurrency or maintains the α ratio for dynamic KV data. This work aims to extract the additive HBM+host bandwidth for LLM inference, in addition to scaling capacity. In our solution, all data accesses from SMs should be demand loads concurrently to both tiers and in proportion to their bandwidth ratio, while performing no data migration at runtime, as prefetching consumes bandwidth. Ideally, we want to make minimal changes, especially avoiding intrusive modifications to performance-critical kernels in LLM serving systems.

Table 1: Summary of comparisons of current and proposed schemes. All schemes add host capacity but fail to provide per-wave concurrency and do not maintain the α ratio for dynamic KV data. [†]No GPU/LLM implementation exists (§5).

Property	Migration		Placement		Goal
	Prefetching [55]	cudaMalloc Managed	BAPP-R [†] [4]	Weighted interleave [†] [28]	
Use host capacity	✓	✓	✓	✓	✓
Migration-free	✗	✗	✓	✓	✓
Proportional accesses	✗	✗	✓	✓	✓
Per-wave concurrency	✗	✗	✗	✗	✓
Works for dyn. data	✗	✗	✗	✗	✓
Key limitation	HBM writes	HBM thrashing	Ratio variance	Access unaware	—

3 Fully Utilizing System Memory Bandwidth

To achieve our goal, we propose BOOST, a runtime system that performs concurrent access to host memory and HBM to accelerate LLM serving. The core idea behind BOOST is to leverage access patterns in LLM kernels to guide page placement and runtime data management decisions. BOOST requires addressing three key questions. §3.1: Developing a page placement policy that allows for concurrent data access without access ratio variance (Fig. 6-(a)) or kernel code changes. §3.2: Finding the correct granularity of access pattern at the kernel that guides decisions. §3.3: Handling important LLM kernels (like attention) that access dynamically provisioned data such as KV pairs.

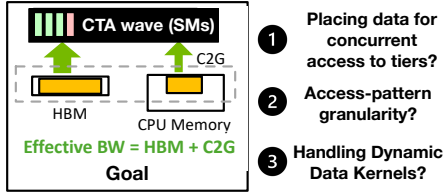


Figure 5: Key questions in enabling concurrent and proportional memory access to fully utilize system bandwidth.

Concurrent and Proportional Access (CAP). We introduce the bandwidth-optimal access pattern ‘CAP’ (Fig. 6-(b)). CAP requires partitioning accesses such that *exactly* $\alpha/(1+\alpha)$ of them are to host while $1/(1+\alpha)$ are to HBM to ensure bandwidth proportionality. If access ratio to host memory is higher than α , we might experience a slowdown. Moreover, CAP requires performing these accesses *concurrently*. Thus, both memory tiers saturate at every wave, eliminating temporal variance to fully utilize system bandwidth.

We first measure the gain that CAP accesses deliver when kernel code is freely editable. We handcraft the add kernel of §2.6 to perform CAP: the data resides in both tiers, a fixed subset of warps reads from the host tier, and the remaining warps read from HBM (writes stay HBM-local). Fig. 6-(c) sweeps the host share of accesses. The speedup peaks at 7.4% at a host share of 8.3%, the measured optimum near the proportional share $\alpha/(1+\alpha) = 9.1\%$, and incurs 6.7% slowdown host ratio considerably exceeds proportional share. The peak of 7.4%, against the 10.4% ideal, is the practical CAP bound for this system because it is difficult for the SMs to saturate both tiers at once (§6.1). Next, we approach this bound for complex LLM serving systems without kernel modifications or data duplication.

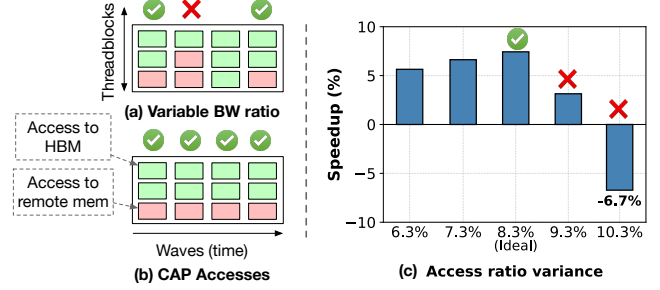


Figure 6: (a) Variable access ratio with BAPP-R. (b) Deterministic access ratio with CAP accesses. (c) Add-kernel speedup at varying access ratios.

3.1 Wave-aware Modulo Page Placement

We could enable CAP by maintaining two pointers for HBM- and host-resident data and editing the kernel. We implement this route in a handwritten matmul kernel, which maps host memory with `cudaHostRegister` and fetches an α fraction of weights from the host tier. The handwritten kernel gains 7.3%, 99% of the CAP bound, so kernel editing extracts nearly the full benefit. However, two properties of decode kernels make per-kernel editing impractical. The matmul path that dominates dense-decode runtime dispatches to the closed-source cuBLAS library [12]. The open kernels are numerous and specialized: vLLM ships 92 custom CUDA kernels [31], FlashAttention-3 compiles 401 forward-pass instantiations, and CUTLASS generates its GEMM kernels from templates. A CAP edit recurs across every instantiation and forces re-autotuning after tile-level changes. Prior serving and memory systems adopt transparency as a design goal [25], as does BOOST. Thus, our **first challenge** is performing CAP without kernel changes.

Page placement must be aware of the access pattern of each wave, which we term *wave-awareness*. Expert and MLP kernels use contiguous virtual memory for model weights and assign each threadblock a virtually contiguous region. Each wave accesses tens of megabytes, spanning tens of 2MB pages (§3.2). To hold an exact ratio across these few pages, we use **modulo-based page placement (MPP)**: for an HBM-to-host bandwidth ratio of $K = 1/\alpha$, MPP allocates the first K pages to HBM and the $(K+1)^{th}$ page to the host tier. For $\alpha = 10\%$, MPP matches α exactly with 11 pages.

Insight-1: Modulo-based page placement (MPP) maintains the exact bandwidth ratio at every threadblock wave, eliminating the ratio variance of random placement (Table 1).

To analyze MPP, we measure the execution time of the Llama-3.3 70B matmul kernel (fused_gate_up CUTLASS kernel [1]) across batch sizes. Fig. 7 plots the average, minimum, and maximum speedup of MPP and BAPP-R on GH200 (§5) over 20 runs, with a portion of data now in the host tier. MPP outperforms BAPP-R with practically no variance, whereas BAPP-R varies widely and slows execution on average in every case (up to 25%). The average MPP speedup matches the maximum BAPP-R speedup, so MPP delivers deterministic bandwidth improvement over random allocation.

MPP is per-kernel, unlike Linux weighted interleaving that stripes data across kernel boundaries, so individual kernels can receive

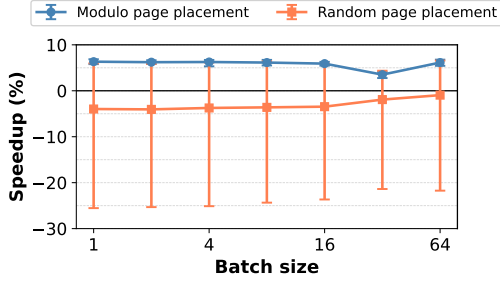


Figure 7: Performance of random and modulo-based page placement for Llama3.3-70B fused gate+up projection matmul kernel (GH200 GPU, details in §5.1) at different batch sizes. Modulo-based page placement consistently outperforms random page placement.

a host share above α (§2.6). Thus, BOOST preserves per-kernel proportional access by allocating each weight tensor separately, so the placement pattern restarts at every tensor and kernel’s host data stays bounded by α to maximize performance.

3.2 CTA Specialization for CAP Accesses

Complex GPU kernels set the number of warps and threadblocks (CTAs) at runtime. Our **second challenge** is choosing the granularity that guides page placement: the thread, warp, or CTA level.

Characterization. Fig. 8 plots the CTA size and grid size of the Qwen3-Next 80B kernel on GH200 across the ShareGPT [50] and LongWriter [6] datasets. The CTA size is 4 or 8 warps, with low variance, whereas the grid size and active CTAs per wave vary widely. Every configuration runs 264K active threads per wave (132 SMs $\times$ 2K threads). Thread- or warp-level granularity might therefore seem best, given their low size variance.

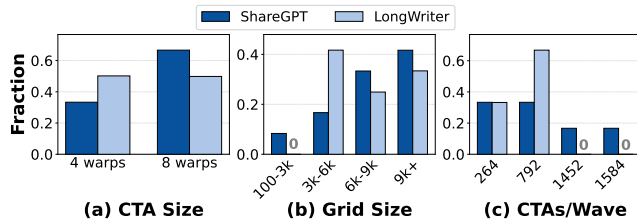


Figure 8: Profiling of Qwen3-Next 80B’s expert compute kernel (GH200 GPU) for ShareGPT and LongWriter datasets. Active threads are 264K, CTA size variation is low, but CTAs per wave vary widely depending on problem size.

Fig. 9 shows the Llama-3.3 70B up-projection performance under each granularity. Thread-level partitioning introduces warp divergence and prevents memory-request coalescing. Warp specialization, where warps within a CTA target different tiers, is too coarse: a 4-warp CTA represents only bandwidth ratios in multiples of 0.25, and widening the CTA to 16 warps refines control but degrades the kernel. Confining a subset of SMs to host accesses (‘4-CTA/SM’) lowers memory-level parallelism and underutilizes HBM. CTA specialization avoids these failures: hundreds to thousands of CTAs run per wave (Fig. 8), giving fine control, and CTAs execute independently on SMs without synchronization overhead.

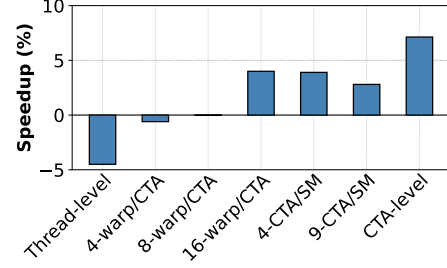


Figure 9: Speedup of Llama-70B up-projection on GH200 compared to HBM-only with different access partitioning.

Concurrency requires that we confine host accesses to a subset of CTAs. When the accesses of a single CTA span both tiers, the host loads stall the SM and delay the co-issued HBM loads, so the SM loses HBM bandwidth. The slowdown is shown with thread-level partitioning in Fig. 9. Restricting host accesses to an α fraction of CTAs instead lets the remaining CTAs saturate HBM and overlap with the latency-bound CTAs.

Insight-2: Guiding page placement on memory tiers using CTA wave accesses facilitates CAP for typical kernels.

This insight also keeps each CTA on a single tier for its lifetime: an $\alpha/(1+\alpha)$ fraction of the CTAs fetch from the host tier exclusively. CTA-level placement gains 7.1% (Fig. 9), 96% of the 7.4% CAP bound compared to 99% for the handwritten kernel (§3.1). Placement alone recovers the gain of kernel editing, obviating kernel changes.

Even with CTA-level placement, the footprint of a CTA need not align to page boundaries: on the Llama-70B down-projection, the footprint of one CTA spans 1.75 pages (§2.6), so under MPP one of these pages could land on the host tier which splits CTA-level accesses, incurring slowdown. For such kernels, BOOST allocates at the granularity of *superpages*: contiguous page groups sized to the least common multiple of the CTA footprint and the page size, so CTA footprint does not span a superpage boundary. BOOST places each superpage in one tier and applies MPP across superpages, limiting CTA-level accesses to one memory tier (§4).

3.3 Wave-aware Page-to-Data Mappings

Modulo page placement works for expert and MLP kernels because their threadblocks access contiguous, statically-allocated weights. Enabling CAP for attention is difficult because serving systems use paged attention [30], which inserts a free pool of KV pages between virtual memory and threadblock data. As requests arrive and complete, the pool provisions and frees KV pages, and the churn makes the mapping from virtual pages to the active wave near-random over time, even under MPP. Our **third challenge** is maintaining CAP despite this dynamic mapping.

The problem is that the placement is fixed at allocation, while the free pool reassigns pages to data at runtime (§2.6). Static allocation-time policies (MPP, BAPP-R, Weighted Interleaving) are unable to dictate a page-to-data mapping that changes dynamically. The pages a wave touches are decided by the allocation order of the free pool, so an interleave that is proportional over the address range still loses the per-wave ratio once blocks churn.

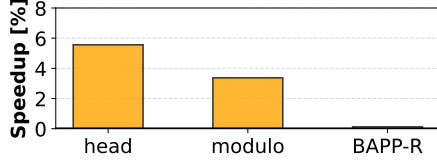


Figure 10: Speedup of FlashAttention3 [49] kernel for Llama3.1-8B on GH200 for different mappings. Managing page-to-KV head chunk mapping maximizes TPOT speedup.

CAP for dynamic data therefore needs awareness of the data structures the kernel accesses, beyond kernel- and CTA-awareness. Placing an α fraction of whole requests on the host tier would satisfy proportionality, but unknown context lengths could let long requests land on the host tier and raise tail latency, so we partition at a finer granularity. We observe that recent attention kernels [30, 49, 61] assign each KV head to different threadblocks and chunk the context of a head across further threadblocks, so the CTAs in a wave process different KV head chunks.

Insight-3: Attention kernel threadblocks operate on ‘KV head chunks’, and mapping α fraction of chunks to pages in host tier at runtime facilitates CAP accesses.

Dynamic-data kernels need two levels: linear placement for average proportionality, and a separate *runtime fulfillment mechanism* that maintains the per-wave ratio under churn—which static interleaving techniques cannot provide (the dynamic-data row of table 1). The runtime fulfillment maps chunks greedily to keep an α fraction of active chunks on the host tier at runtime. As Fig. 10 shows, this chunk-aware policy outperforms MPP and BAPP-R, yielding end-to-end TPOT speedup even when only the attention kernel performs CAP. Head-chunk mapping gains 5.9% on FlashAttention-3 (81% of the CAP bound), and the fulfillment holds the host fraction within $\pm 1\%$ of the ideal ratio at runtime (§5). We address practical challenges (fragmentation, tensor reshaping) in §4.5. Fig. 11 summarizes the key insights of our design.

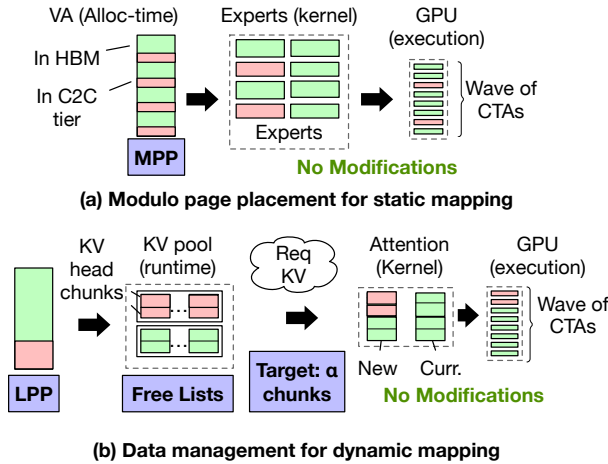


Figure 11: Top: MPP ensures CAP with static data (model weights). Bottom: Wave-aware LLM system for CAP with dynamic KV data. LPP is linear (BW-proportional) placement.

4 BOOST Design and Implementation

BOOST enables CAP accesses on every wave of LLM serving without kernel changes. It places data across both HBM and host memory, so it increases the usable capacity over HBM-only serving in addition to raising bandwidth utilization. Every access is a cache-coherent demand load to one tier, and BOOST performs no data migration at runtime and is simple to implement.

4.1 Design Overview

Fig. 12 shows the two components of BOOST. The Page Allocation Manager (PAM) places pages at allocation time and realizes the wave-aware placement of §3.1 and §3.2. The Data Shape Remapper (DSR) manages dynamic KV data at runtime and realizes the runtime fulfillment of §3.3. Prior heterogeneous-memory work [4] and CPU-NUMA systems [28] place pages bandwidth-proportionally at the memory allocator and the device driver. These layers observe only the address space. CAP additionally requires awareness of kernel access patterns and runtime management of dynamic data structures (§3), and only the serving system has this information, because it loads the model weights and manages the KV pool. BOOST therefore implements both components inside the serving system. Both components run on the CPU, and BOOST adds no GPU code.

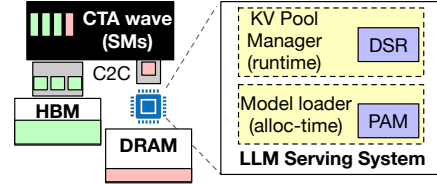


Figure 12: Design overview of BOOST.

4.2 Page Allocation Manager (PAM) Design

PAM resides in the allocation path of the serving system and places pages according to the data each kernel accesses. For statically allocated data, such as the model weights of expert and MLP kernels, PAM applies modulo page placement (MPP) across the two tiers, and uses superpages for kernels whose CTA footprint misaligns with pages (§3.2). For dynamically mapped data, such as the KV pairs of the attention kernel, allocation-time placement alone cannot hold the per-wave ratio (§3.3), so PAM provides the first of the two management levels. PAM applies linear page placement (LPP), which partitions the virtual address space contiguously: the first $1/(\alpha + 1)$ of pages map to HBM and the last $\alpha/(\alpha + 1)$ map to the host tier (Fig. 11-bottom left). Because LPP makes the tier of a page a fixed function of its virtual address, DSR controls the physical placement of a KV block by choosing its virtual address. PAM overrides the default allocation routines of the serving system, enabling kernels to perform CAP accesses transparently.

4.3 Data Shape Remapper (DSR) Design

DSR provides the second management level for dynamic data. DSR extends the runtime of the serving system so that the per-wave ratio is maintained approximately rather than exactly. The KV block manager of the serving system requests and frees *blocks* of KV pairs, and each block holds a fixed number of pairs (often 16). DSR maintains a bandwidth-proportional access ratio at every wave

through two mechanisms. First, DSR maps the KV head chunks within each block to the two tiers through LPP-allocated virtual memory. Second, DSR fulfills block requests so that an α fraction of the active KV head chunks stays on the host tier.

Block-level KV Head Chunk Partitioning. DSR partitions KV data along two axes—attention heads and requests. Kernels assign different KV heads to different threadblocks, but many LLMs expose too few heads for fine control. Qwen3-30B, for example, has four KV heads, and placing one head on the host tier already sends 25% of the KV accesses there, far above the $\alpha = 10\%$ target. DSR therefore defines two block types. A *single-head-host* block maps one head to the host tier and the remaining heads to HBM. An *all-local* block maps all heads to HBM. For a target of $1/M$ of the KV data on the host tier with H heads, DSR maintains single-head-host and all-local blocks in the ratio H to $M - H$, so the host-resident share of KV data averages $1/M$ over time.

Wave-aware Block Fulfillment. As requests arrive and complete, the allocator tracks the distribution of free blocks to hold the target ratio. When a new request needs KV cache, DSR assigns the single-head-host type if the free single-head-host fraction exceeds H/M , and assigns the all-local type otherwise. This greedy policy keeps the host-fetched fraction of KV head chunks near α across request lengths and arrival patterns. All blocks of a request share one type, which reduces temporal variance across waves.

The greedy policy meets two boundary conditions when a free list empties. When the single-head-host list empties, DSR assigns the all-local type and the host fraction falls below α . The system forfeits part of the bandwidth benefit but incurs no slowdown, because only a fraction above α oversubscribes the host tier (§3). When the all-local list empties, DSR examines the number of free single-head-host blocks. Typically, the shortage stems from an imbalance in request lengths, and DSR appends single-head-host blocks to a request of the all-local type. The assignment is sticky, so the request continues to receive single-head-host blocks and the mixing of block types stays confined to that request. When total available blocks are low and the system faces critical memory pressure, DSR defers to the scheduler of the serving system.

4.4 PAM Implementation Details

Superpages. MPP at page granularity assumes the weight footprint of each CTA aligns to a page boundary. In practice, a CTA computes an output tile of 64–256 columns, and each column spans the full contraction dimension of the weight. The contraction dimension of some kernels is a non-power-of-2 value, such as 28K on the Llama down-projection, so each FP8 column occupies 28KB and the footprint of one CTA reaches 3.5MB, which spans page boundaries (§3.2). For such kernels, PAM sizes superpages to the least common multiple of the per-CTA footprint and the page size (e.g., $\text{LCM}(3.5\text{MB}, 2\text{MB}) = 14\text{MB}$ for Llama-70B down-projection), places each superpage in one tier, and applies MPP across superpages to bound the host fraction to α relative to HBM.

Interleaving on Cache-Coherent Systems. We implement PAM in vLLM’s `model_loader`. Grace Hopper is cache-coherent, so the OS exposes GPU memory as a NUMA node and standard Linux APIs interleave CPU and GPU memory in one contiguous virtual range. For each 2MB region (`MAP_HUGETLB`, over the 64KB default,

to limit TLB thrashing), PAM calls `mmap` with `mbind` to select the node and `cudaHostRegister` to expose it to the GPU, then zeros it to force physical backing. For MPP, PAM maps K superpages to HBM and one superpage to the host tier in turn, and LPP allocations are similarly bandwidth-proportional. When $K = 1/\alpha$ is not an integer, PAM sets $K = \lceil 1/\alpha \rceil$, which keeps the host fraction below α ; a fraction above α oversubscribes the host tier and incurs slowdown (§3). A small residual variance remains, because the final wave of a kernel holds fewer CTAs than the rest. However, eliminating this variance requires the wave count in advance, which are not currently exposed by parameterized CUTLASS kernels.

4.5 DSR Implementation Details

Virtual Memory Integration. DSR extends the KV block manager to maintain two free lists, one for each block type. The two lists exploit the existing virtual memory indirection of the serving system to control physical placement, so DSR requires no per-page tracking and no changes to the allocation logic of the kernels.

Tensor Layout Reordering. A single KV block, holding 16 tokens across all layers and heads, occupies approximately 2.5MB in models like Llama-3.3 70B-FP8. The block size misaligns with the 2MB page size, so the heads of one block straddle page boundaries and DSR cannot place a single head on the host tier directly. DSR resolves the misalignment by reordering the tensor layout. The paged attention kernels of vLLM describe tensors using base pointers, dimension lengths, and strides, which permits non-contiguous virtual memory as long as the access pattern stays regular. DSR reshapes the KV tensor to place heads as the outermost dimension, so the data of each head becomes contiguous and aligns to the LPP partition. The reordered layout preserves the locality optimizations of the kernel within each head and bounds the host access ratio by $1/H$ even under maximum imbalance. When $H > M$, DSR mixes blocks with $\lfloor H/M \rfloor$ and $\lceil H/M \rceil$ host heads to maintain the target ratio.

5 Evaluation Results

5.1 Methodology

Serving System. We extend vLLM (v0.17.0) and our changes are approximately 800 LoC (Python and PyTorch).

Models. We evaluate two recent LLMs for our main evaluations: Qwen3-Next-80B MoE [53] and Llama3.3-70B [18] dense LLM. We also evaluate the larger Qwen2.5-72B [47], for scalability study with tensor parallelism. Unless specified, all models are in FP8.

Workloads. We use synthetic workloads with varying context lengths. For the dense model we use an input/output length of 256/2K, 512/4K, 1K/8K. For the MoE model we use a smaller input/output length of 64/512, 128/1K, 256/2K. We also show sensitivity to increasing context length (§5.9) and batch sizes (§5.10).

Experimental Setup. We use a Grace Hopper system (GH200) with 96GB HBM and 480GB CPU memory. Peak GPU HBM bandwidth is 3.63TiB/s and CPU memory bandwidth is 476GiB/s. GPU connects to the CPU memory tier via cache-coherent NVLink C2C at 419GiB/s. We measure peak HBM and C2G bandwidth at 3330GiB/s and 350GiB/s, respectively ($\alpha=10\%$). All main evaluations use the Grace Hopper system. We evaluate systems with higher α of 12% using H100-based emulation of the host tier (details in §5.12).

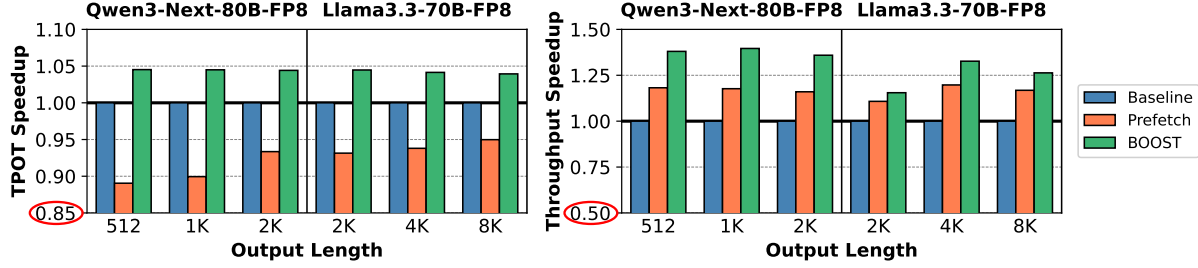


Figure 13: Left: TPOT speedup for Llama3.3 dense LLM (batch size of 10) and Qwen3-Next MoE (batch size of 88) for low latency serving, normalized to HBM-only baseline. BOOST improves TPOT by 4.3% on average (geometric mean) while prefetching degrades TPOT by 6% for iso-batch serving. Right: Normalized throughput speedup in high throughput serving, where batch size scales to fill the available memory capacity. BOOST provides 31% average throughput speedup, 15% higher than prefetching.

Serving Scenarios and Metrics. All experiments use prefill-decode disaggregation, the standard production configuration [42, 54]. Our GH200 node has a single GPU, so the prefill and decode instances time-share the GPU, and we report the metrics of the decode instance. We evaluate all systems under two representative LLM serving scenarios (§5.5 provides the performance decomposition):

- **Low-Latency Serving.** The goal is to minimize time-per-output-token (TPOT). We fix the maximum runnable batch size (batch of 88 requests for Qwen3-Next and 10 requests for Llama-70B) and measure TPOT as the average time to generate each output token during the decode phase. The batch size is fixed, so any TPOT change stems solely from the additional system bandwidth utilization.
- **High-throughput Serving.** The goal is to maximize tokens per second (TPS) throughput. The serving system automatically adjusts the batch size to fully utilize the available memory capacity. We measure TPS as the total number of tokens generated (including the first token) divided by the total execution time. The auto-scaled batch lets the added capacity of the host tier raise concurrency, so TPS captures the capacity and bandwidth benefits together.

Evaluated Policies.

- **Baseline.** The baseline system allocates all data in HBM without using C2G memory. The setup represents standard LLM serving when the model fits in the GPUs and does not utilize the additional host tier memory bandwidth.
- **Prefetch.** Asynchronous prefetching in vLLM [55] stores one out of every K layers in host memory ($1/K$ approximates $1/(1 + \alpha)$) and asynchronously prefetches it to HBM before use (we use $K = 12$ to maximize speedup).
- **BOOST.** We evaluate our solution that employs MPP for static model weights and LPP with runtime α management for dynamic KV data. The placement modulus is 12: one of every 12 pages resides on the host tier, a host share of 8.3% that matches the measured-optimal share of §3.

We additionally evaluate CUDA’s driver-based migration policy, `cudaMallocManaged`. Finally, we evaluate BAPP-R and Lin-uxed weighted interleaving policies, representing SoTA bandwidth-aware page placement [4, 11, 28].

5.2 Key Results: TPOT and Throughput Speedup

Fig. 13 plots normalized TPOT for low-latency serving (left) and system throughput for high-throughput serving (right) across Qwen3-Next and Llama-3.3 at varying context lengths, normalized to the HBM-only baseline. At the same batch size, BOOST improves TPOT by 4.3% on average. Prefetching degrades TPOT by 6%, making BOOST 11% faster. BOOST provides consistent TPOT benefit for both dense and MoE models. Prefetching, however, suffers more for the MoE: dynamic expert activation patterns introduce variability in layer execution time, causing prefetches to miss their timing window. This leads to a 9.3% average TPOT degradation for the MoE. BOOST is unaffected because it performs cacheline-granular demand access to both tiers concurrently, requiring no orchestration of data movement, which has the added benefit of reducing implementation complexity and finetuning requirements.

BOOST improves system throughput by 31% on average, and up to 40%, as the 10% additional capacity from the host tier substantially increases the runnable batch size. The added capacity drives most of this throughput gain; §5.5 isolates the bandwidth-concurrency component at about 4%. Prefetching improves throughput by 17% on average but suffers from data migrations that consume HBM read bandwidth. Throughput improvement of Qwen3-Next 80B is higher because it is more capacity-constrained at 80GB out of 96GB taken by model weights, so only about 16GB is available for KV cache. By improving effective GPU capacity by 10%, or about 9.6GB, BOOST provides 60% additional space for KV, while also increasing system bandwidth. BOOST is functional for all vLLM kernels that access model weights or KV pairs, including FlashAttention-3 [49] and closed-source cuBLAS routines [12] used by MLP kernels.

5.3 Memory Bandwidth Utilization

We measure the average bandwidth utilization in the low-latency serving setup with the 4K-context workload. Using Nvidia Nsight Systems [40], we measure average utilization of the decode phase of a request. Fig. 14 shows the total bandwidth utilization (left) and normalized HBM and C2G bandwidth utilization for various policies. Prefetching degrades the demand bandwidth utilization available to the SMs by 7.8% compared to HBM-only serving, as writes due to data migration contend with demand reads over the simplex HBM interface. BOOST improves the system bandwidth utilization by 3% over HBM-only serving and by 12% over prefetching.

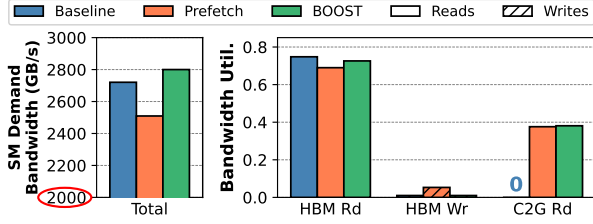


Figure 14: Bandwidth utilization (left) and its breakdown (right) of HBM and C2G for Llama-3.3 (512/4K context). BOOST improves bandwidth utilization (HBM+C2G) by 3% over HBM-only baseline and by 12% over prefetching.

5.4 Performance Contribution Breakdown

We isolate the contributions of PAM and DSR using system throughput: PAM alone on the expert and MLP kernels (BOOST-MLP-only) versus the full system (BOOST), which adds DSR to the attention kernel. Table 2 reports throughput for Llama-3.3 70B, normalized to HBM-only at the maximum runnable batch of 10. The expert and MLP kernels hold most of the decode time in large models, so PAM alone delivers 1.23x. Adding DSR for attention yields a further 7.2% (1.32x total). DSR contributes less than PAM because attention occupies roughly 30% of decode runtime (Nsight); model weights dominate, and PAM serves them. The contribution of DSR—9 of the 32 points in Table 2, or 28% of the total gain—is bounded near the runtime share of attention. This share grows at long context, where the attention footprint grows relative to the weights. Overall, the two optimizations are additive.

Table 2: Performance of BOOST for Llama-3.3 70B (512/4K in/out context) with PAM (BOOST-MLP-only) and with both PAM+DSR (BOOST). Our optimizations are additive.

Model/ Config	HBM-only	BOOST (MLP-only)	BOOST
Llama-3.3 70B	1026	1263 (1.23x)	1352 (1.32x)

5.5 Isolation of Capacity and Bandwidth Gains

BOOST raises throughput two ways: the host tier enlarges the runnable batch, and concurrent access to both tiers raises demand bandwidth. Isolating them requires an HBM-only baseline with the same total capacity as BOOST. Lacking a GH200 above 96GB, we run BOOST below full HBM utilization and grant the HBM-only baseline equal total capacity entirely in HBM (table 3). For Llama-3.3 70B at 1K/8K context, this capacity-matched baseline raises throughput by 55% over the HBM-only reference, a capacity component available to any unified-memory mechanism. At equal capacity and batch, BOOST improves both throughput and TPOT by a further 4%. This bandwidth-concurrency component is unique to BOOST; no capacity-only mechanism provides it (table 4), and it is consistent with the 4.3% iso-batch TPOT gain of Fig. 13.

5.6 Comparison: BAPP-R

We implement BAPP-R by performing bandwidth-proportional and random page allocation for both model weights and KV pairs. As Fig. 15 shows for Llama-3.3 and Qwen3-Next models for low-latency

Table 3: BOOST speedup decomposition for Llama-3.3 70B (1K/8K). The capacity-matched baseline holds the same capacity as BOOST in HBM, isolating bandwidth-induced gains.

Configuration	Capacity (BS)	Throughput (TPS)	TPOT (ms)
HBM-only (85% HBM mem-util)	81.6GB (BS=6)	222	27.1
HBM-capacity (95% HBM)	91.2GB (BS=10)	344 (1.55x)	29.0
BOOST (85% HBM + CPU-mem)	91.2GB (BS=10)	359 (1.62x)	27.9

setting, BAPP-R causes a 0.4% and 2.7% TPOT degradation for the MoE and dense LLM, respectively. In contrast, BOOST provides consistent TPOT gains of 4.5% and 4.2% for the MoE and dense LLM. Random placement suffers from the ratio variance of table 1—the access ratio to the two tiers varies across waves, which degrades bandwidth utilization. We measure the average memory bandwidth utilization for BAPP-R using Nsight Systems (not plotted for brevity) and find that it degrades demand bandwidth to the SMs by 3.6% compared to HBM-only serving and by 6.4% compared to BOOST. The lower bandwidth utilization results in worse TPOT for BAPP-R.

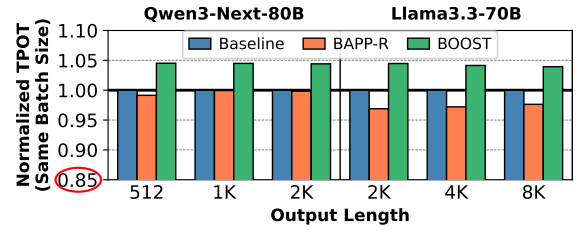


Figure 15: TPOT speedup of BOOST and BAPP-R for Qwen3-Next and Llama3.3. BAPP-R provides 0.4% and 2.7% TPOT degradation for MoE and dense LLM, respectively.

5.7 Comparison: OS and CUDA Schemes

We evaluate OS and CUDA capacity-expansion schemes in table 4. Weighted interleave provides proportionality and raises throughput by 30%, but lacks concurrency and fails to improve iso-batch TPOT (0.99), while BOOST delivers 35% throughput and 4% TPOT speedup at the 512/4K configuration of table 4. Weighted interleave splits the footprint of a CTA across tiers and stripes across kernel boundaries (§2.6). BOOST resolves these problems with superpages and per-tensor MPP (§3.2, §3.1). BOOST confines the accesses of each CTA to one tier through MPP and superpages (§4) and bounds the host fraction of accesses by every kernel to at most α .

Proportional interleave maps contiguous pages to each tier, so the accesses of early waves target HBM and those of late waves target the host tier; this reduces to on-demand fetching and drops performance to 0.6x on the up-projection, a 67% slowdown. cudaMallocManaged migrates pages toward the accessing processor: it matches HBM-only when the working set fits in HBM (1.00), but once the set exceeds HBM the migrations thrash HBM as a page cache, and each access reduces to an on-demand host fetch (throughput of 0.19x, a 5x slowdown). BOOST performs no migration; it pins placement through mbind and lets the SM issue 64B cache-coherent demand loads to both tiers.

Table 4: Comparison of capacity expansion techniques using Llama3.3-70B (512/4K). Existing techniques fail to deliver consistent speedup (per-row maximum in bold). All four techniques (columns) use identical batch sizes.

Llama Norm. perf	Weighted interleave	Proportional interleave	cudaMallocManaged	BOOST
up-proj kernel	1.06	0.60	1.00	1.07
down-proj kernel	0.99	1.00	1.00	1.03
Iso-batch TPOT	0.99	0.75	1.00	1.04
Max throughput	1.30	1.08	0.19	1.35

5.8 Impact on Tail Latency

Fig. 16 plots the P99 inter-token latency (ITL), the 99th percentile of the per-token latency during decode, against request rate for BOOST and BAPP-R (Llama-3.3 70B, LongWriter). Requests arrive with Poisson inter-arrival times. BOOST holds a lower P99 ITL than BAPP-R at every serviceable request rate, by 3–13% (7.8% geometric mean), reflecting its higher demand bandwidth. Both systems share the same host-tier capacity and saturate near 0.55 req/s, so under a fixed P99 ITL service level agreement, BOOST sustains a higher request rate. The primary advantage of BOOST over BAPP-R is lower per-token latency at each request rate.

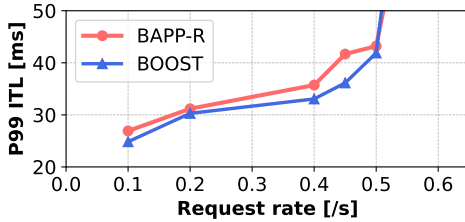


Figure 16: P99 ITL versus request rate for BOOST and BAPP-R on the decode instance (Llama-3.3 70B, LongWriter). BOOST reduces P99 ITL by 7.8% (geometric mean) over BAPP-R across the serviceable load range.

5.9 Context Length Variation

We evaluate Llama-3.3 model with longer output context lengths and plot throughput speedup in Fig. 17 normalized to HBM-only serving. We fix the (synthetic) input context to 1K. BOOST consistently outperforms prefetching across all context lengths by 11% on average. BOOST’s speedup drops slightly from 25% at 16K context to 19% at 32K context. Overall, BOOST scales to longer context workloads, making it viable for reasoning and agentic systems.

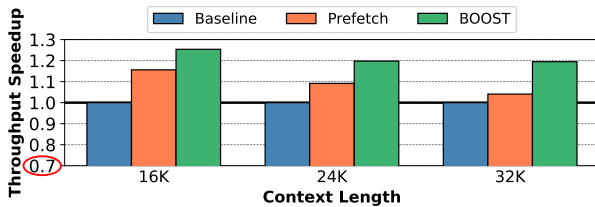


Figure 17: Normalized throughput speedup of Llama3.3 model with longer context (output fixed at 8K). BOOST scales to long context workloads.

5.10 Sensitivity: Batch Size

Fig. 18 plots kernel-level speedup at a fixed 64/512 input/output context across batch sizes. The Qwen3-Next MoE slows by 2% at batch 16: at low batch, few experts activate, so few threadblocks launch and the access ratio becomes imbalanced. The slowdown vanishes as the batch grows, and BOOST reaches roughly 6% speedup at batch 256 and sustains it at 1024, where all 512 experts activate (each decode token routes to 10 experts plus one shared). The dense Llama-3.3 shows the opposite trend: speedup falls from 7% at batch 8 to 6% at batch 64, and turns into a 0.5% slowdown at batch 256. Grace Hopper does not L2-cache loads from GPU to host [16], so at large batch a weight tile is split across threadblocks that compute subsets of the batch; inter-threadblock reuse at L2 then matters, host loads bypass it, and BOOST loses performance.

Out-of-the-box (OOTB) models. We evaluate two more models to confirm these regimes: Qwen3-30B MoE and DeepseekCoder-33B [20] dense model. The MoEs incur slowdown at batch 16, where only a few experts activate, and gain as the batch grows, reaching 3.7–6.2% at batch 256. The dense models gain at low batch sizes but start to incur slowdown at batch 256, where multiple CTAs share a weight tile and host loads bypass the L2.

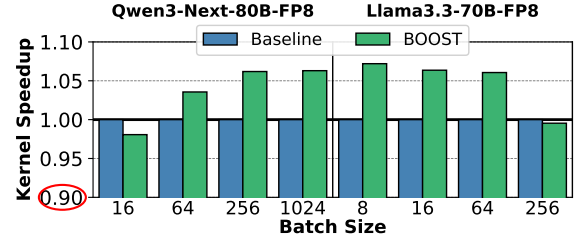


Figure 18: Speedup of expert kernel from a MoE model (Qwen3-Next) and matmul kernel from a dense model (Llama3.3) across different batch sizes.

5.11 Performance of DSR’s Approximate CAP

DSR holds the host fraction of KV cache accesses near the ideal ratio at runtime, even when a model exposes few KV heads. Fig. 19 plots the realized host fraction over a Qwen3-30B run (four KV heads) on LongWriter at 2.5 req/s, against the configured ideal of 8.3% (one of every 12 pages on the host tier, §5.1). The rolling median stays between 7.7% and 9.2% throughput, within $\pm 1\%$ of the ideal ratio, and the rolling min–max band stays equally narrow. The worst-case granularity bound is $1/H = 25\%$ (§4), yet the realized deviation is just 1%, an order of magnitude below this bound, because the block-type mix of DSR spreads single-head-host blocks across requests. DSR thus approaches the determinism of MPP for dynamic KV data, despite the indirection of the free pool.

5.12 Bandwidth Ratio Ablation with NVLinks

We emulate a higher bandwidth ratio ($\alpha = 12\%$) with two NVLink-connected H100 GPUs (80GB each); NVLink provides cache-coherent load-stores with latency and caching behavior similar to the Grace CPU [16]. The LLM runs on one H100, and the second serves only its memory, measured at 2.81TiB/s HBM and 351GiB/s over NVLink. Fig. 20 compares the expert (Qwen3-Next) and matmul (Llama-3.3)

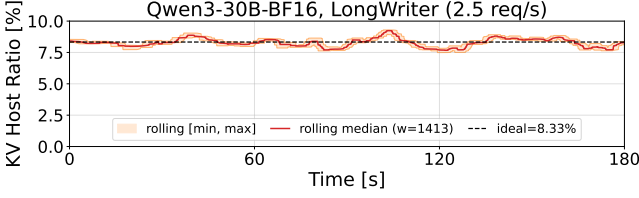


Figure 19: DSR keeps the host ratio of KV cache near ideal despite low number of attention heads ($H=4$).

kernel speedups on GH200 and the H100 emulation. BOOST reaches 9.6% average latency speedup on the H100 versus 6.3% on GH200, because the H100 offers a higher α . BOOST thus benefits systems with higher-bandwidth secondary tiers.

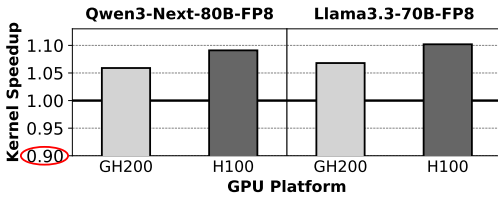


Figure 20: Speedup of expert kernel from a MoE model (Qwen3-Next) and matmul kernel from a dense model (Llama3.3) across different hardware platforms. Input activations use batch size of 256 (MoE) and 64 (dense). H100-emulation provides better speedup due to higher α .

5.13 Larger Models with Tensor Parallelism

Our GH200 node is single-GPU, so we extend the H100 emulation (§5.12) to evaluate scalability on larger models with tensor parallelism (TP). Fig. 21 shows TPOT speedup normalized to HBM-only for Qwen2.5-72B (batch 6) and Llama3.3-70B (batch 4) at TP=2 for low-latency serving, both in BF16. BOOST improves TPOT by 6.5% on average, extending to multi-GPU systems.

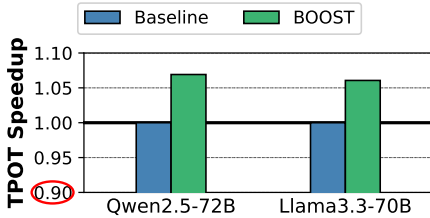


Figure 21: TPOT speedup normalized to HBM-only serving for larger models with tensor parallelism (TP=2) on H100-emulation. BOOST improves TPOT by 6.5% on average.

6 Discussion

6.1 Future Hardware Design

Scaling BOOST to larger future batches needs two things. First, the GPU must L2-cache host loads, as in prior works [35, 62], to retain performance under high inter-threadblock reuse (§5.10). Second, the SMs must saturate both tiers at once: on an H200, HBM saturation uses 84% of the SMs and the host tier needs another 20%, exceeding

the SM budget, whereas an H100 needs only 62% for HBM. Future systems must provision enough SMs and buffers to drive both tiers.

Performance Model. To project BOOST onto future systems, we build an analytical throughput model. Rather than slow cycle-level simulation or coarse roofline bounds, it models LLM operators at tens-to-hundreds of microseconds, shards their FLOPs and bytes by parallelization scheme, and maps them onto hardware. It captures heterogeneous memory through a critical-path timing model with *slack* analysis that accounts for prefetching across L2 and HBM. We implement it in Python. We validate it against published GB200 NVL72 measurements of DeepSeek-V3/R1 decode [54] (2K context, HBM-only, DP+EP). It predicts peak per-GPU decode throughput within -6.0% in FP8 (8,539 vs 9,087 tok/s/GPU) and $+4.9\%$ in a mixed FP4 configuration with FP8 attention (14,041 vs 13,386 tok/s/GPU).

The Grace Blackwell (GB200) [38] pairs two GPUs per CPU ($\alpha = 3\%$); future Vera-class systems [8] reach 900GB/s C2G bandwidth (α up to 44%). Fig. 22 projects BOOST and idealized prefetching on GB200 for DeepSeek R1 [19] in FP4 at 128K context (TP=2), scaling α from 2.5% to 40%, normalized to HBM-only. BOOST gains rise with α , approaching 3x and exceeding prefetching by up to 60%, so BOOST scales to future systems.

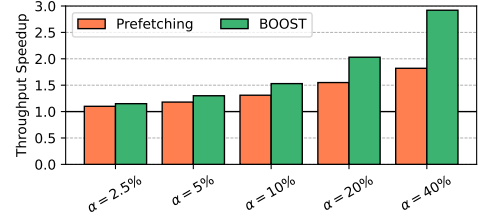


Figure 22: Projected normalized throughput speedup of BOOST and prefetching for GB200 running DeepSeek R1 FP4 at 128K context/user at varying α (high-TPS setting). BOOST outperforms prefetching by up to 60%.

6.2 Applicability of BOOST

Generality. BOOST remains useful even when the working set fits entirely in HBM, unlike prefetching which lowers demand and bandwidth. The host tier holds idle bandwidth regardless of capacity pressure, and BOOST converts this bandwidth into a 4.3% iso-batch TPOT gain (Fig. 13); the added capacity then arrives as a further benefit when workloads grow (§5.5). BOOST also requires no kernel changes, so unmodified models benefit out of the box (§5.10). Finally, our design assumes only a cache-coherent secondary tier, making BOOST extensible to other tiered memory systems.

Decoding Regimes. BOOST targets the memory-bandwidth-bound decode instances, and two regimes limit its gains. First, BOOST degrades when many CTAs share one weight tile, because host loads bypass the L2 [16] and break the reuse. Hopper caps the MLP output tile at $N=128$, so beyond batch 128 (256 on Blackwell) multiple CTAs share a tile, as in prefill. BOOST therefore confines placement to decode, placing a fraction of weights on the host tier under co-location; L2 caching of host loads [35, 62] would lift this limit (§6.1). Second, BOOST loses at small MoE batches, where few experts activate and too few CTAs launch to hide the latency of host-routed

CTAs: Qwen3-Next slows 2% at batch 16 (Fig. 18), but the slow-down vanishes with scale, and BOOST gains 6% at the kernel level at batch 1K. To sustain consistent gains, we recommend decode batch sizes of 64–1K for MoE and 8–128 for dense models (up to 256 on Blackwell owing to dedicated TMEM for output tiles).

Interconnects. BOOST is functional wherever the GPU issues load-stores to a host tier. Our H100 emulation (Fig. 20) realizes a multi-GPU NVLink configuration, where one GPU serves memory for another. BOOST also runs on PCIe, but its low bandwidth ratio ($\alpha \approx 2\%$) yields marginal gains, as we expect for CXL. High-bandwidth NVLink-attached and disaggregated memory are the strongest fit on future high- α systems (§6.1).

6.3 BOOST Design Considerations

System integration. BOOST places its mechanisms in the serving system because that layer alone sees the access pattern of each kernel and the allocation of each tensor that CAP requires; the OS is agnostic to kernel boundaries (§2.6), and cuBLAS [12] admits no kernel edits. BOOST modifies only the weight loader and KV-pool manager, assigns pages through `mmap` and `mbind`, and performs no migration: PAM places all pages statically while the kernels issue demand loads, and DSR changes only the assignment of KV data to those pages at runtime (§4). It adds no GPU fragmentation, since vLLM partitions HBM into fixed KV blocks at startup [31], and reserves 9.6GB (2% of CPU capacity) in 1GB pages.

Requirements for transparent CAP. BOOST obviates kernel changes by relying on three properties of LLM kernels. First, cache-coherent load-stores let the SM reach the host tier with ordinary loads, so BOOST needs no message passing or in-kernel orchestration. Second, high-performance kernels such as paged attention and tiled GEMMs address virtual tensors and ignore physical placement, so BOOST reassigns pages through `mmap` and `mbind` and reconstructs the same tensor without changing the access pattern. Third, tiling, per-CTA resource use, and TMA produce a steady per-CTA access stream, so BOOST places each CTA on one tier and routes an α fraction to the host tier. BOOST does not split a CTA across tiers, which stalls the SM (§3); it interleaves across CTAs at page granularity. The handwritten kernel of §3.1 beats BOOST by 0.2 points (7.3% versus 7.1%), too little to justify editing various kernels and models against the single 800-LoC change of BOOST.

7 Related Work

7.1 Page Management for GPU Memory System

Agarwal et al. [3] migrate CPU data to HBM with one-touch access and nearby-page prefetching, regulating aggressiveness by bandwidth utilization. Such migration reduces to linear placement, which slows our add kernel by 72% (LPP). BATMAN [11] controls migration direction to hold a target local-access rate, preserving proportionality. LLM inference has predictable access patterns, and *any* migration spends HBM bandwidth on writes, limiting gains; BOOST migrates nothing and places pages directly. Agarwal et al. [4] show full bandwidth needs bandwidth-proportional placement (BAPP-R). We are the first to show that both proportionality *and* concurrency are required, which BAPP-R fails to provide.

7.2 CPU Tiered Memory Management

Nimble [60] improves page-migration throughput in the OS kernel. Adaptive Migration Policy [22] selects a migration policy from application behavior. Soar [34] guides migration with loaded latency and memory-level parallelism beyond page hotness. Colloid [56] places pages proactively to minimize loaded access latency, and HeteroOS [27] extends cooperative placement into the hypervisor to cut migrations. These policies suit latency-sensitive CPUs with unknown access patterns, but they treat memory as a hierarchy rather than peers, so they incur migration overhead and do not provide bandwidth-proportional placement, similar to serving systems.

7.3 GPU Memory Capacity Expansion

FlexGen [51] and ZeRO-Offload [48] offload model weights and optimizer state to CPU memory for training on memory-constrained GPUs. For inference, MoE-Lightning [9], PRESERVE [65], Pie [58], and others [10, 29, 59, 64] prefetch weights and KV data to HBM, scheduling migration to overlap with computation; others offload to SSD [43] or speculate [71], and MoonCake [46] pools CPU memory across nodes for warm KV state such as prefix cache. These schemes expand capacity but treat the tiers hierarchically and expend HBM write bandwidth. They target the compute-bound high-throughput regime, where spare compute hides migration latency. BOOST instead treats the tiers as peers with SM-issued cache-coherent loads, achieving additive bandwidth in the bandwidth-bound online regime, where no compute slack exists. InfiniGen [32] and NEO [26] are orthogonal, reducing the volume of KV fetched and offloading attention computation to the CPU respectively, while BOOST accelerates the demand traffic across both tiers.

7.4 LLM Serving Systems

Frameworks such as vLLM [31], TRT-LLM [41], and SGLang [68] layer many optimizations. Kernel optimizations—FlashAttention [13, 14, 49], FlashDecode [23], FlashInfer [61], operator fusion [37, 45, 66, 70], and megakernels [57]—tile caches and cut memory traffic and launch overhead, raising compute efficiency; they complement BOOST, which raises bandwidth utilization. Scheduling optimizations such as continuous batching [63], prefill chunking [5], and KV migration [52], and memory schemes such as LLM-aware allocation [67] and PagedAttention [30], remain HBM-centric. vAttention [44] removes non-contiguous virtual memory via CUDA VMM and complements BOOST: it removes the reshaping in DSR while PAM makes page management CAP-compliant.

8 Conclusion

LLM decode is memory-bandwidth-bound, yet current serving systems underutilize the host memory bandwidth. Extracting additive bandwidth of both tiers requires *concurrent and bandwidth-proportional accesses (CAP)* within every wave, which existing placement and migration schemes do not provide. We present BOOST, a runtime system that performs CAP through wave-aware page placement for static weights and runtime KV-pool management for dynamic KV data, without kernel changes. BOOST improves TPOT by 4.3% where prefetching degrades it by 6%, and raises throughput by 31% on average—with 4% bandwidth-improvement that only BOOST’s concurrent access provides—15% above prefetching.

Acknowledgments

We thank the reviewers of ISCA-2026 and MICRO-2026 for their valuable feedback, and Prof. Tushar Krishna for providing access to Grace Hopper node through Georgia Tech’s CRNCH Cluster for initial experiments. Anish was partly funded by Nvidia Graduate Fellowship. This work was supported by NSF grant 233304.

References

- [1] 2026. <https://docs.nvidia.com/cutlass/latest/>
- [2] Deepak Bhaskar Acharya, Karthigeyan Kuppan, and B Divya. 2025. Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access* (2025).
- [3] Neha Agarwal, David Nellans, Mike O’Connor, Stephen W Keckler, and Thomas F Wenisch. 2015. Unlocking bandwidth for GPUs in CC-NUMA systems. In *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 354–365.
- [4] Neha Agarwal, David Nellans, Mark Stephenson, Mike O’Connor, and Stephen W Keckler. 2015. Page placement strategies for GPUs within heterogeneous memory systems. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*. 607–618.
- [5] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [6] Yushi Bai, Jiajie Zhang, Xin Lv, Linzhi Zheng, Siqi Zhu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongWriter: Unleashing 10,000+ Word Generation from Long Context LLMs. *arXiv preprint arXiv:2408.07055* (2024).
- [7] Sean Betts. 2023. Peering inside GPT-4: Understanding its mixture of experts (MOE) architecture. <https://medium.com/@seanbetts/peering-inside-gpt-4-understanding-its-mixture-of-experts-moe-architecture-2a42eb8dbcb3>
- [8] NADDOD Blog. [n.d.]. Nvidia Vera Rubin NVL144: Next-generation high-performance computing platform. https://www.naddod.com/blog/nvidia-vera-rubin-nvl144-next-generation-high-performance-computing-platform?srsltid=AfmBOopp_TZf23xSujA5PGTZOh6p3s2g9yXlgh7CMn6LjSrl2-JAsFZN
- [9] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E Gonzalez, Matei Zaharia, and Ion Stoica. 2025. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 715–730.
- [10] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, et al. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 1014–1029.
- [11] Chiachen Chou, Aamer Jaleel, and Moinuddin Qureshi. 2017. BATMAN: Techniques for maximizing system bandwidth of memory systems with stacked-DRAM. In *Proceedings of the International Symposium on Memory Systems*. 268–280.
- [12] NVIDIA Corporation. [n.d.]. https://docs.nvidia.com/cuda/pdf/CUBLAS_Library.pdf
- [13] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [14] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [15] Hugging Face. 2025. Large Language Model Text Generation Inference. <https://github.com/huggingface/text-generation-inference>
- [16] Luigi Fusco, Mikhail Khalilov, Marcin Chrapek, Giridhar Chukkappalli, Thomas Schulthess, and Torsten Hoefler. 2024. Understanding data movement in tightly coupled heterogeneous systems: A case study with the Grace Hopper superchip. *arXiv preprint arXiv:2408.11556* (2024).
- [17] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 415–430.
- [18] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [19] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [20] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).
- [21] John Hennessy, David Patterson, and Christos Kozyrakis. 2025. *Computer Architecture: a Quantitative Approach, 7th edition*. Morgan Kaufmann.
- [22] Taekyung Heo, Yang Wang, Wei Cui, Jaehyuk Huh, and Lintao Zhang. 2020. Adaptive page migration policy with huge pages in tiered memory systems. *IEEE Trans. Comput.* 71, 1 (2020), 53–68.
- [23] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhang Dong, and Yu Wang. 2024. Flashdecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. *Proceedings of Machine Learning and Systems* 6 (2024), 148–161.
- [24] Yitao Hu, Xiulong Liu, Guotao Yang, Linxuan Li, Kai Zeng, Zhixin Zhao, Sheng Chen, Laiping Zhao, Wenxin Li, and Keqiu Li. 2025. TightLLM: Maximizing Throughput for LLM Inference via Adaptive Offloading Policy. *IEEE Trans. Comput.* (2025).
- [25] Wenhao Huang, Zhaolin Duan, Laiping Zhao, Yuhao Zhang, Yanjie Wang, Yiming Li, Yihan Wang, Yichi Chen, Zhihang Tang, Kang Chen, Deze Zeng, Wenxin Li, and Keqiu Li. 2026. uShare: Non-Intrusive Kernel Co-Locating on NVIDIA GPUs. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1–14. <https://doi.org/10.1109/HPCA68181.2026.11408561>
- [26] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2025. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *Proceedings of Machine Learning and Systems* 7 (2025).
- [27] Sudarsan Kannan, Ada Gavrilovska, Vishal Gupta, and Karsten Schwan. 2017. Heteros: Os design for heterogeneous memory management in datacenter. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*. 521–534.
- [28] Michael Kerrisk. [n.d.]. Set_mempolicy(2) - linux manual page. https://man7.org/linux/man-pages/man2/set_mempolicy.2.html
- [29] Rui Kong, Yuanchun Li, Weijun Wang, Linghe Kong, and Yunxin Liu. 2025. Serving MoE models on resource-constrained edge devices via dynamic expert swapping. *IEEE Trans. Comput.* (2025).
- [30] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [31] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [32] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.
- [33] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [34] Jinshu Liu, Hamid Hadian, Hanchen Xu, and Huaicheng Li. 2025. Tiered memory management beyond hotness. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 731–747.
- [35] Ugljesa Milic, Oreste Villa, Evgeny Bolotin, Akhil Arunkumar, Eiman Ebrahimi, Aamer Jaleel, Alex Ramirez, and David Nellans. 2017. Beyond the socket: NUMA-aware GPUs. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (Cambridge, Massachusetts) (MICRO-50 ’17)*. Association for Computing Machinery, New York, NY, USA, 123–135. <https://doi.org/10.1145/3123939.3124534>
- [36] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393* (2025).
- [37] Nandeeeka Nayak, Xinrui Wu, Toluwanimi O Odemuyiwa, Michael Pellauer, Joel S Emer, and Christopher W Fletcher. 2024. Fusemax: Leveraging extended einsums to optimize attention accelerator design. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1458–1473.
- [38] NVIDIA. 2024. NVIDIA GB200 NVL72. <https://www.nvidia.com/en-us/data-center/gb200-nvl72/>
- [39] NVIDIA. 2024. Nvidia Grace Hopper superchip architecture whitepaper. <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-hopper>
- [40] Nvidia. 2025. Nvidia Nsight System User Guide. <https://docs.nvidia.com/nsight-systems/UserGuide/index.html>
- [41] Nvidia. 2025. TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. <https://github.com/NVIDIA/TensorRT-LLM>
- [42] LMSYS Org. [n.d.]. Deploying DeepSeek with PD disaggregation and large-scale expert parallelism on 96 h100 gpus. <https://lmsys.org/blog/2025-05-05-large-scale-ep/>

- [43] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2025. InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1510–1525.
- [44] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vattention: Dynamic memory management for serving llms without pagedattention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1133–1150.
- [45] Le Qin, Junwei Cui, Weilin Cai, and Jiayi Huang. 2025. Chimera: Communication Fusion for Hybrid Parallelism in Large Language Models. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 498–513. <https://doi.org/10.1145/3695053.3731025>
- [46] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading more storage for less computation—a {KVCACHE-centric} architecture for serving {LLM} chatbot. In *23rd USENIX conference on file and storage technologies (FAST 25)*. 155–170.
- [47] Qwen, ., An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yaqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2025. Qwen2.5 Technical Report. [arXiv:2412.15115 \[cs.CL\]](https://arxiv.org/abs/2412.15115) <https://arxiv.org/abs/2412.15115>
- [48] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 551–564.
- [49] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.
- [50] ShareGPT. 2023. ShareGPT. <https://www.kaggle.com/datasets/roschildrui/sharegpt-v3-unfiltered-cleaned-split>. Accessed: 2025-10-10.
- [51] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [52] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llmunix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 173–191.
- [53] Qwen Team. 2025. Qwen3 Technical Report. [arXiv:2505.09388 \[cs.CL\]](https://arxiv.org/abs/2505.09388) <https://arxiv.org/abs/2505.09388>
- [54] The SGLang Team. 2025. Deploying DeepSeek on GB200 NVL72 with pd and large scale EP (part II): 3.8X prefill, 4.8x Decode throughput - LMSYS blog. <https://www.lmsys.org/blog/2025-09-25-gb200-part-2/>
- [55] Meta vLLM Team and NVIDIA. 2026. Driving Vllm Wideep and large-scale serving toward maturity on Blackwell (Part I). <https://vllm.ai/blog/dsr1-gb200-part1>
- [56] Midhul Vuppapapati and Rachit Agarwal. 2024. Tiered Memory Management: Access Latency is the Key!. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 79–94.
- [57] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. 2025. Mirage: A Multi-Level Superoptimizer for Tensor Programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, Boston, MA.
- [58] Yi Xu, Ziming Mao, Xiangxi Mo, Shu Liu, and Ion Stoica. 2024. Pie: Pooling cpu memory for llm inference. *arXiv preprint arXiv:2411.09317* (2024).
- [59] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2025. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache. [arXiv:2401.14361 \[cs.LG\]](https://arxiv.org/abs/2401.14361) <https://arxiv.org/abs/2401.14361>
- [60] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacharjee. 2019. Nimble page management for tiered memory systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 331–345.
- [61] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *arXiv preprint arXiv:2501.01005* (2025).
- [62] Vinson Young, Aamer Jaleel, Evgeny Bolotin, Eiman Ebrahimi, David Nellans, and Oreste Villa. 2018. Combining HW/SW Mechanisms to Improve NUMA Performance of Multi-GPU Systems. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 339–351. <https://doi.org/10.1109/MICRO.2018.00035>
- [63] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.
- [64] Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. 2025. Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading. [arXiv:2502.05370 \[cs.LG\]](https://arxiv.org/abs/2502.05370) <https://arxiv.org/abs/2502.05370>
- [65] Ahmet Caner Yüzügüler, Jiawei Zhuang, and Lukas Cavigelli. 2025. PRESERVE: Prefetching Model Weights and KV-Cache in Distributed LLM Serving. *arXiv preprint arXiv:2501.08192* (2025).
- [66] Yujia Zhai, Chengquan Jiang, Leyuan Wang, Xiaoying Jia, Shang Zhang, Zizhong Chen, Xin Liu, and Yibo Zhu. 2023. Bytetransformer: A high-performance transformer boosted for variable-length inputs. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 344–355.
- [67] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, et al. 2025. JENGA: Effective memory management for serving LLM with heterogeneity. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 446–461.
- [68] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
- [69] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.
- [70] Donglin Zhuang, Zhen Zheng, Haojun Xia, Xiafei Qiu, Junjie Bai, Wei Lin, and Shuaiwen Leon Song. 2024. {MonoNN}: Enabling a New Monolithic Optimization Space for Neural Network Inference Tasks on Modern {GPU-Centric} Architectures. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 989–1005.
- [71] Xiangwen Zhuge, Xu Shen, Zeyu Wang, Fan Dang, Xuan Ding, Danyang Li, Yahui Han, Tianxiang Hao, and Zheng Yang. 2025. SpecOffload: Unlocking Latent GPU Capacity for LLM Inference on Resource-Constrained Devices. *arXiv preprint arXiv:2505.10259* (2025).